

FLIPKART E-COMMERCE DATABASE SYSTEM

Requirement Analysis & Database Design

Student Name: Maria Antony Selvam S

Course: 2nd BCA-A

College: Kamaraj College, Thoothukudi

Project based on a Flipkart-style e-commerce platform. This academic project models customers, sellers, products, orders, order items and payments.

1. Project Overview

The Flipkart E-Commerce Database System is designed to manage the major activities of an online shopping platform. Customers can register, browse products, place orders and make payments. Sellers can list products and maintain product information. The database stores customer, seller, product, order and payment information in a structured way.

2. E-Commerce Business Process

1. Customer registration/login → 2. Product browsing → 3. Product selection → 4. Order placement → 5. Payment processing → 6. Order confirmation → 7. Delivery/status tracking → 8. Order completion or cancellation. Sellers are responsible for maintaining their product listings and prices.

3. Customer Management Requirements

The system should store a unique customer ID, name, email, phone number and address. Email and phone should be unique where applicable. The system should allow customer information to be inserted, viewed, updated and deleted while maintaining database integrity.

4. Major Entities

Customer, Seller, Product, Orders, Order_Item and Payment are the major entities. Order_Item is included because one order may contain multiple products and a product may appear in many orders.

5. Entities and Attributes

Customer: Customer_ID (PK), Name, Email (UNIQUE), Phone (UNIQUE), Address.

Seller: Seller_ID (PK), Seller_Name, Email (UNIQUE), Phone (UNIQUE), Business_Name.

Product: Product_ID (PK), Seller_ID (FK), Product_Name, Category, Price, Stock, Rating.

Orders: Order_ID (PK), Customer_ID (FK), Order_Date, Status, Total_Amount.

Order_Item: Order_Item_ID (PK), Order_ID (FK), Product_ID (FK), Quantity, Unit_Price.

Payment: Payment_ID (PK), Order_ID (FK), Payment_Date, Payment_Method, Amount, Payment_Status.

6. Relationships Between Entities

A Customer can place many Orders (1:M). A Seller can sell many Products (1:M). An Order can contain many Order_Items (1:M). A Product can occur in many Order_Items (1:M). An Order can have one or more payment records depending on the business process; this academic design uses one payment record per order (1:1).

7. ER Diagram

Use the following structure to draw the ER diagram in Word/Canva/draw.io:

CUSTOMER (Customer_ID PK) 1 ■■■■ M ORDERS (Order_ID PK, Customer_ID FK)

SELLER (Seller_ID PK) 1 ■■■■ M PRODUCT (Product_ID PK, Seller_ID FK)

ORDERS 1 ■■■■ M ORDER_ITEM (Order_Item_ID PK, Order_ID FK)

PRODUCT 1 ■■■■ M ORDER_ITEM (Order_Item_ID PK, Product_ID FK)

ORDERS 1 ■■■■ 1 PAYMENT (Payment_ID PK, Order_ID FK)

8. Functional Requirements

- Add and manage customers.
- Add and manage sellers.
- Add and manage products.
- Maintain product price and stock.
- Place customer orders.
- Store individual products in each order.
- Record payment details.
- Search and display order information.
- Update order status.
- Delete records while respecting foreign-key relationships.
- Generate order-summary information using SQL views.

9. Non-Functional Requirements

The database should provide data consistency, integrity, reasonable performance and secure handling of customer information. Primary keys should uniquely identify records, foreign keys should maintain relationships, and UNIQUE/NOT NULL constraints should prevent invalid duplicate or missing data.

10. Review Questions & Answers

- Q1. What is a primary key? A primary key uniquely identifies each row in a table.
- Q2. What is a foreign key? A foreign key links a column to a primary key in another table.
- Q3. Why is Order_Item required? It resolves the many-to-many relationship between Orders and Products.
- Q4. What is a UNIQUE constraint? It prevents duplicate values in a column.
- Q5. What is a view? A view is a virtual table based on a SQL query.
- Q6. What is CRUD? Create, Read, Update and Delete operations.
- Q7. Why use NOT NULL? It ensures required fields cannot be left empty.

11. Sample Program & Expected Output

Run the supplied SQL script in MySQL Workbench. Example SELECT output:

Customer_Name	Order_ID	Order_Status	Total_Amount
---------------	----------	--------------	--------------

Arun Kumar	1001	Delivered	45998.00
------------	------	-----------	----------

Priya Devi	1002	Shipped	1499.00
------------	------	---------	---------

Kavin Raj	1003	Processing	5999.00
-----------	------	------------	---------

The screenshots required for submission should be captured from your own MySQL Workbench after executing the script. Include screenshots of database creation, tables, INSERT results, SELECT/CRUD queries and views.

12. Submission Checklist

1. Requirement Analysis PDF
2. SQL script file
3. ER diagram inserted in the document
4. Screenshots from MySQL Workbench
5. GitHub repository containing both files
6. Copy the GitHub repository link
7. Submit the link in the Google Form provided by your staff.